

運用生成式 AI 實作 JIRA 使用者故事

來源：<https://codelabs.developers.google.com/genai-for-dev-jira-gemini?hl=zh-tw>

步驟數：10

運用生成式 AI 實作 JIRA 使用者故事

目錄

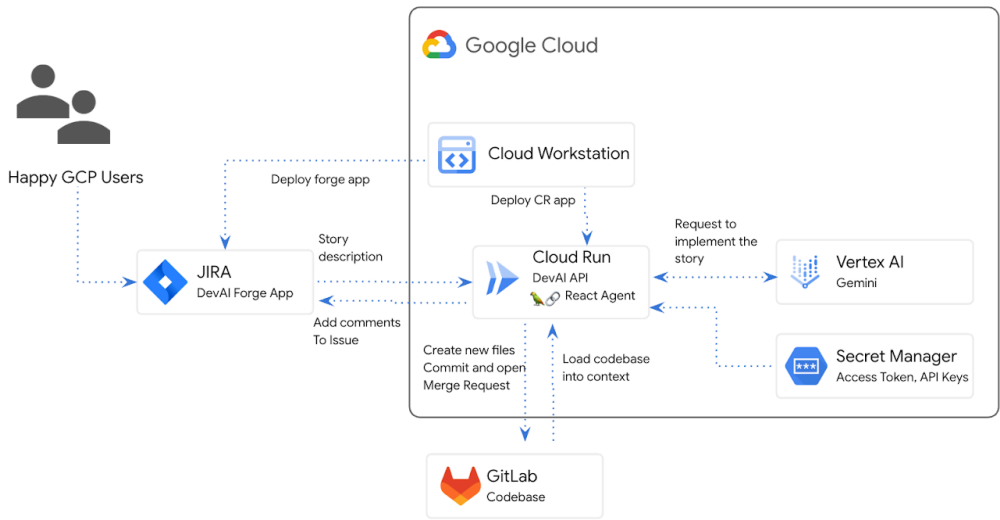
1. [總覽](#)
2. [設定和需求](#)
3. [測試提示的選項](#)
4. [查看應用程式](#)
5. [GitLab 存放區和工具包設定](#)
6. [準備在 Cloud Run 上部署應用程式](#)
7. [將 Devai-API 部署到 Cloud Run](#)
8. [自動導入故事](#)
9. [將變更推送至 GitHub 存放區](#)
10. [恭喜！](#)

1. 總覽

在本實驗室中，您將建構解決方案，運用生成式 AI 自動實作 JIRA 使用者故事。

JIRA user story implementation with GenAI

Gemini + GitLab + JIRA



學習目標

本實驗室主要分為幾個部分：

- 部署 Cloud Run 應用程式，與 Gemini API 整合
- 建立及部署 JIRA 適用的 Atlassian Forge 應用程式
- 使用 LangChain ReAct 代理程式自動執行 GitLab 工作

必要條件

- 本實驗室假設您已熟悉 Cloud 控制台和 Cloud Shell 環境。

步驟 2 · 約 0 分鐘

2. 設定和需求

設定 Cloud 專案

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud console interface. At the top, there is a 'Select a project' dropdown menu. Below it, the 'Select a project' page is visible, featuring a search bar labeled 'Search projects and folders'. To the right of the search bar is a 'NEW PROJECT' button. Below the search bar, the 'New Project' form is shown. It includes a 'Project name' field with the value 'My Project 13475', a 'Project ID' field with the value 'inbound-analogy-389614' (highlighted with a blue box), and a 'Location' field. The 'Project ID' field has a note: 'It cannot be changed later. EDIT'. At the bottom of the form are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

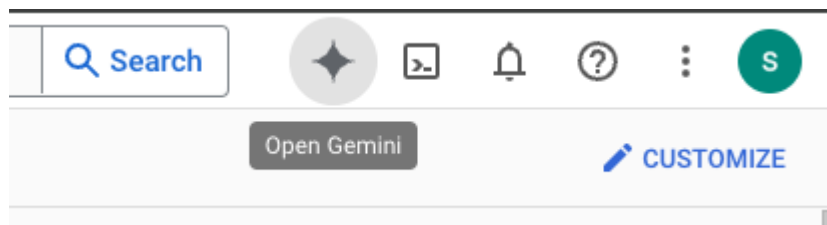
注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

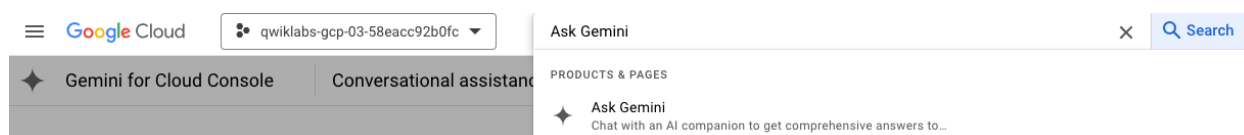
用戶可參加[價值\\$300 美元的免費試用計畫](#)。

環境設定

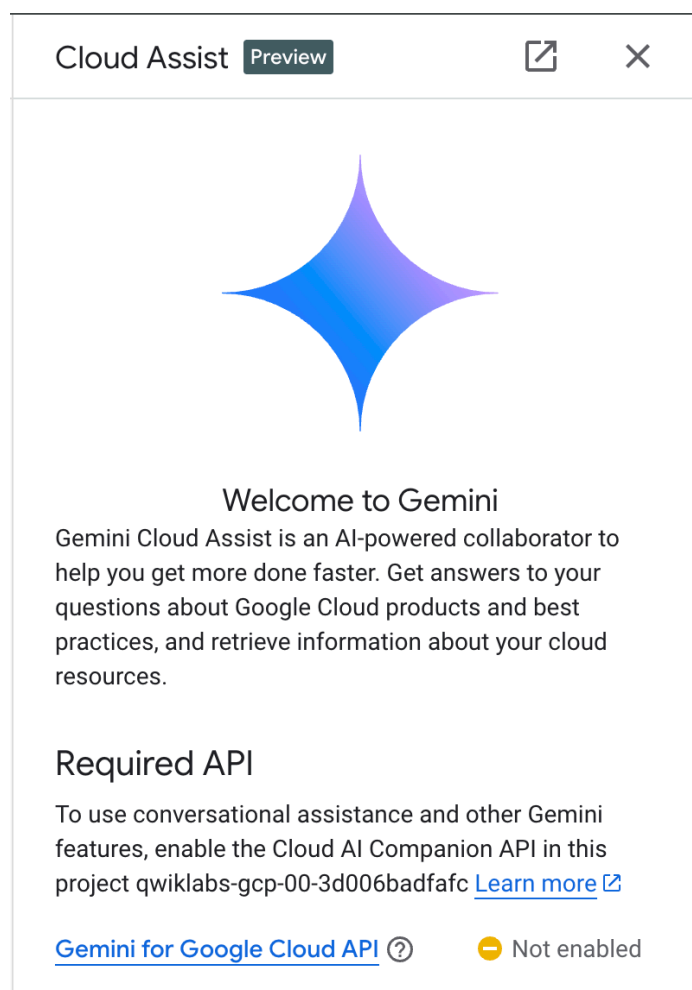
開啟 Gemini 對話。



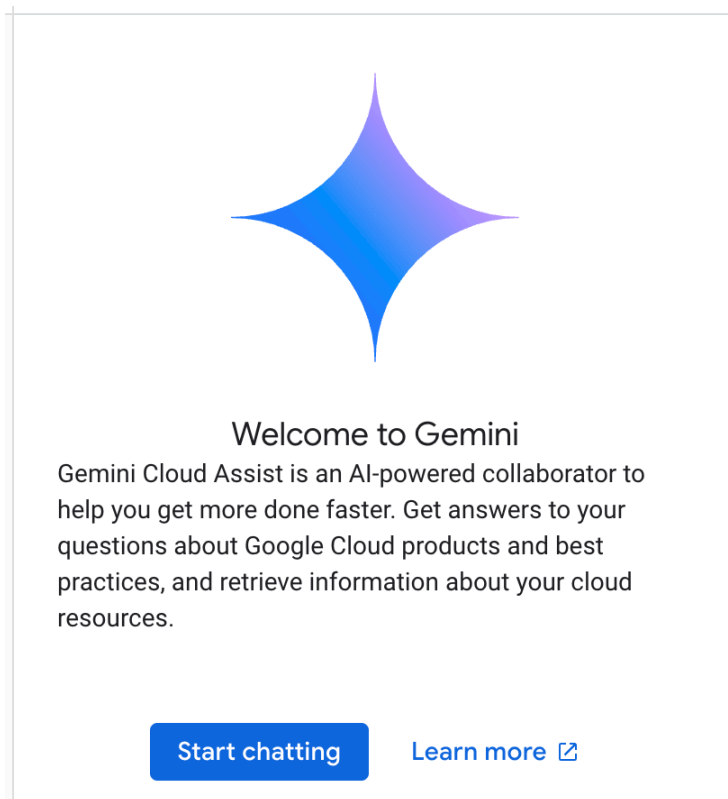
或在搜尋列中輸入「問問 Gemini」。



啟用 Gemini for Google Cloud API：



按一下「`Start chatting`」，然後按照其中一個範例問題操作，或自行輸入提示詞來試用。



建議提示詞：

- 請用 5 個重點說明 Cloud Run。
- 您是 Google Cloud Run 產品經理，請用 5 個簡短重點向學生說明 Cloud Run。
- 您是 Google Cloud Run 產品經理，請用 5 個簡短重點向認證 Kubernetes 開發人員說明 Cloud Run。
- 您是 Google Cloud Run 產品經理，請以 5 個簡短重點，向資深開發人員說明何時該使用 Cloud Run，何時該使用 GKE。

若要進一步瞭解如何撰寫更優質的提示，請參閱[提示指南](#)。

瞭解 Gemini for Google Cloud 如何使用您的資料

Google 對隱私權的承諾

Google 是業界首批發布 [AI/機器學習隱私權承諾](#) 的公司，該文提到我們的信念：除了極致的安全性之外，客戶也應該對儲存在雲端的自家資料保有最大的掌控權。

您提交及收到的資料

您向 Gemini 提出的問題 (包括提交給 Gemini 分析或完成的任何輸入資訊或程式碼)，都稱為提示。您從 Gemini 取得的答案或程式碼完成建議稱為「回覆」。我們不會將您的提示詞或 Gemini 的回覆用做模型訓練資料。

提示加密

當您將提示提交給 Gemini 時，您的資料會在傳輸過程中加密，然後輸入 Gemini 的基礎模型。

Gemini 生成的節目資料

Gemini 是以 Google Cloud 第一方程式碼和精選第三方程式碼訓練而成。您必須對程式碼的安全性、測試和效用負責，包括 Gemini 提供的任何程式碼完成、生成或分析功能。

[進一步瞭解](#) Google 如何處理提示。

步驟 3 · 約 0 分鐘

3. 測試提示的選項

你可以透過多種方式測試提示。

- [Vertex AI Studio](#)

Vertex AI Studio 是 Google Cloud Vertex AI 平台的一部分，專門用於簡化及加速生成式 AI 模型的開發和使用。

- [Google AI Studio](#)

Google AI Studio 是網頁工具，可讓您設計提示工程原型，並透過 Gemini API 進行實驗。

- [Gemini 網頁應用程式](#) (gemini.google.com)

Google Gemini 網頁應用程式 (gemini.google.com) 是一款網頁工具，可協助您探索及運用 Google Gemini AI 模型強大的功能。

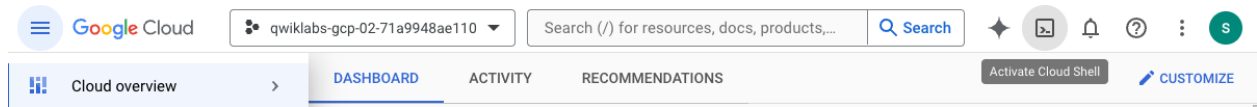
- [Android](#)和 [iOS 版 Google 應用程式](#)
-

步驟 4 · 約 0 分鐘

4. 查看應用程式

建立服務帳戶

返回 Google Cloud 控制台，然後點選搜尋列右側的圖示，啟用 Cloud Shell。



在開啟的終端機中執行下列指令，建立新的服務帳戶和金鑰。

您將使用這個服務帳戶，從 Cloud Run 應用程式對 Vertex AI Gemini API 進行 API 呼叫。

使用 Qwiklabs 專案詳細資料設定專案詳細資料。

範例：`qwiklabs-gcp-00-2c10937585bb`

```
gcloud config set project YOUR_QWIKLABS_PROJECT_ID
```

建立服務帳戶。

```
PROJECT_ID=$(gcloud config get-value project)
SERVICE_ACCOUNT_NAME='vertex-client'
DISPLAY_NAME='Vertex Client'

gcloud iam service-accounts create $SERVICE_ACCOUNT_NAME --project $PROJECT_ID --display-name
"$DISPLAY_NAME"
```

授予角色。

```
gcloud projects add-iam-policy-binding $PROJECT_ID --
member="serviceAccount:$SERVICE_ACCOUNT_NAME@$PROJECT_ID.iam.gserviceaccount.com" --
role="roles/aiplatform.admin"

gcloud projects add-iam-policy-binding $PROJECT_ID --
member="serviceAccount:$SERVICE_ACCOUNT_NAME@$PROJECT_ID.iam.gserviceaccount.com" --
role="roles/aiplatform.user"

gcloud projects add-iam-policy-binding $PROJECT_ID --
member="serviceAccount:$SERVICE_ACCOUNT_NAME@$PROJECT_ID.iam.gserviceaccount.com" --
role="roles/cloudbuild.builds.editor"

gcloud projects add-iam-policy-binding $PROJECT_ID --
member="serviceAccount:$SERVICE_ACCOUNT_NAME@$PROJECT_ID.iam.gserviceaccount.com" --
role="roles/artifactregistry.admin"

gcloud projects add-iam-policy-binding $PROJECT_ID --
member="serviceAccount:$SERVICE_ACCOUNT_NAME@$PROJECT_ID.iam.gserviceaccount.com" --
role="roles/storage.admin"

gcloud projects add-iam-policy-binding $PROJECT_ID --
member="serviceAccount:$SERVICE_ACCOUNT_NAME@$PROJECT_ID.iam.gserviceaccount.com" --
role="roles/run.admin"

gcloud projects add-iam-policy-binding $PROJECT_ID --
member="serviceAccount:$SERVICE_ACCOUNT_NAME@$PROJECT_ID.iam.gserviceaccount.com" --
role="roles/secretmanager.secretAccessor"
```

如果系統提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

啟用必要服務，即可使用 Vertex AI API 和 Gemini 對話。

```
gcloud services enable \
  generativelanguage.googleapis.com \
  aiplatform.googleapis.com \
  cloudaicompanion.googleapis.com \
  run.googleapis.com \
  cloudresourcemanager.googleapis.com
```

啟用必要服務，即可使用 Vertex AI API 和 Gemini 對話。

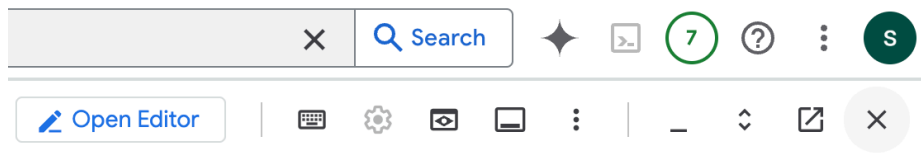
```
gcloud services enable \
  artifactregistry.googleapis.com \
  cloudbuild.googleapis.com \
  runapps.googleapis.com \
  workstations.googleapis.com \
  servicemanagement.googleapis.com \
  secretmanager.googleapis.com \
  containerscanning.googleapis.com
```

複製 GitHub 存放區

複製 GitHub 存放區。

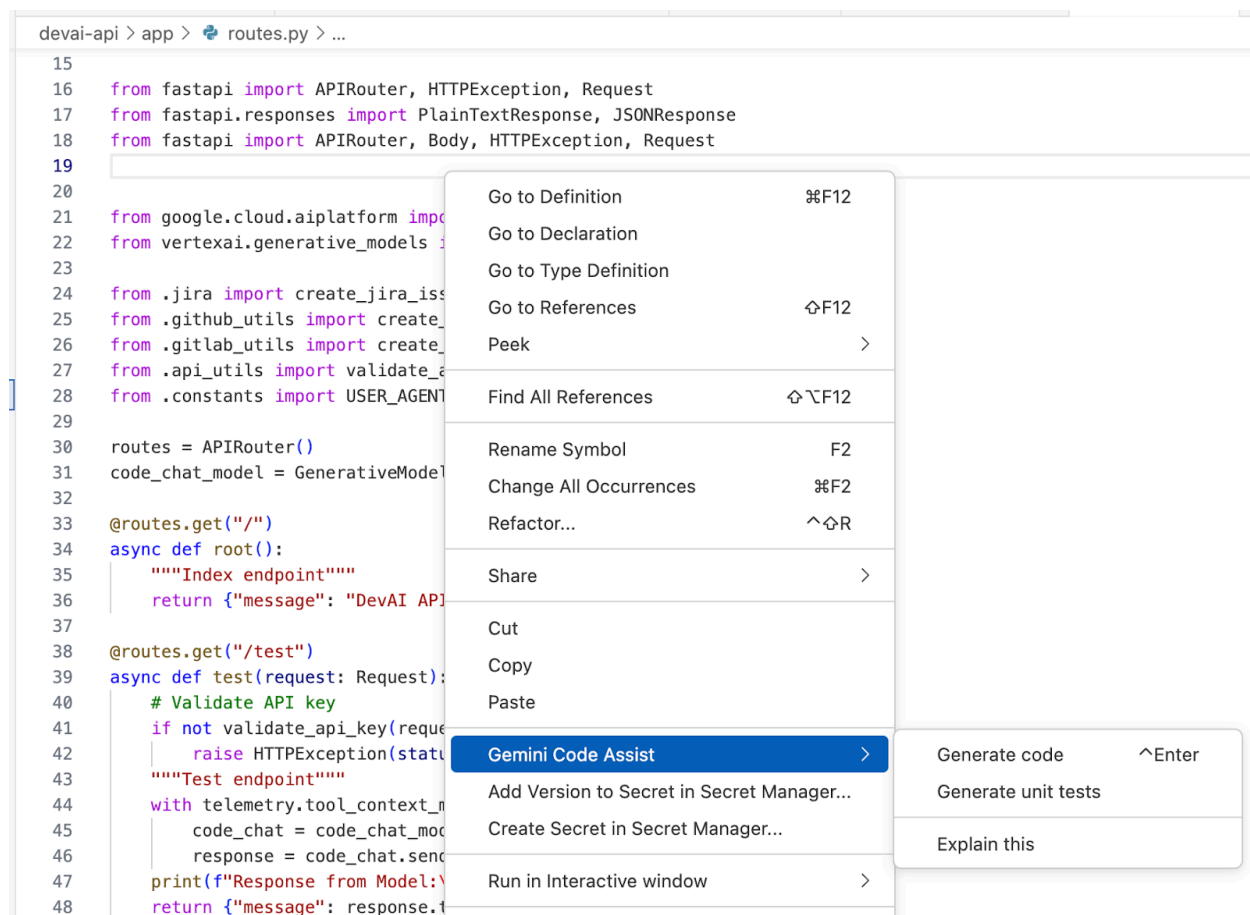
```
git clone https://github.com/GoogleCloudPlatform/genai-for-developers.git
```

開啟 Cloud Shell 編輯器

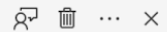


使用「File / Open Folder」選單項目開啟「genai-for-developers」。

開啟「devai-api/app/routes.py」檔案，然後在檔案中的任意位置按一下滑鼠右鍵，並從內容選單中選取「Gemini > Explain」this"。



查看 Gemini 對所選檔案的說明。



 student 701a20b3

Explain this



Gemini

Okay, let's break down the `/home/student_01_ce62a6529c37/genai-for-developers/devai-api/app/routes.py` file, which defines the API routes for a developer-focused Generative AI (GenAI) application.

Overall Purpose

This file (`routes.py`) sets up the different endpoints (URLs) that users can interact with to leverage the GenAI capabilities of the application. It handles requests related to:

- **Testing the API:** A simple test endpoint to check if the service is running.
- **Creating GitHub Pull Requests:** An endpoint to generate and create pull requests on GitHub based on user prompts.
- **Creating GitLab Merge Requests:** An endpoint to generate and create merge requests on GitLab based on user prompts.
- **Generating code and creating a GitLab merge request:** endpoint to generate code based on provided prompt and create a merge request in GitLab.
- **Creating Jira Issues:** An endpoint to generate technical descriptions and create Jira issues based on user prompts.

Key Components and Functionality

1. Imports:

- `fastapi`: The web framework used for building the API.
- `fastapi.APIRouter`, `fastapi.HTTPException`, `fastapi.Request`, `fastapi.Body`: Classes and functions for defining routes, handling errors, processing requests, and handling request bodies.
- `fastapi.responses.PlainTextResponse`, `fastapi.responses.JSONResponse`: Classes for sending text and json responses.
- `google.cloud.aiplatform.telemetry`: For tracking usage and other events (telemetry).
- `vertexai.generative_models.GenerativeModel`: For interacting with Vertex AI's generative models (like Gemini).
- `jira.create_jira_issue`: A function (likely from `app/jira.py`) to create Jira issues.
- `github_utils.create_pull_request`: A function (from `app/github_utils.py`) to create GitHub pull requests.
- `gitlab_utils.create_merge_request`, `gitlab_utils.MergeRequestError`: Functions (from `app/gitlab_utils.py`) to create GitLab merge requests and handle related errors.
- `api_utils.validate_api_key`: A function (from `app/api_utils.py`) to check the validity of API keys.
- `constants.USER_AGENT`, `constants.MODEL_NAME`: Constants for the application's user agent and the name of the generative model.

2. `routes = APIRouter()`:

- Creates an `APIRouter` instance, which will be used to define the API endpoints. This is a way to organize routes in FastAPI.

3. `code_chat_model = GenerativeModel(MODEL_NAME)`:

- Initializes the generative model (likely Gemini) using the name defined in `constants.py`. This model will be

步驟 5 · 約 0 分鐘

5. GitLab 存放區和工具包設定

在本實驗室中，您將使用 [GitLabToolkit](#) 自動建立 GitLab 合併要求。

LangChain 工具包總覽

[LangChain 工具包](#)是一組工具，旨在簡化及提升 LangChain 應用程式的開發作業。這些工具包提供各種功能，但通常可協助您：

- **連結外部資料來源**：在 LangChain 應用程式中存取及整合來自 API、資料庫和其他外部來源的資訊。
- **進階提示技巧**：使用預先建立的提示詞或自訂提示詞，提升與語言模型的互動體驗。
- **建立及管理鏈結**：輕鬆建構複雜的鏈結，並有效管理。
- **評估和監控**：分析 LangChain 應用程式和鏈結的效能。

熱門的 LangChain 工具包包括：

- **代理執行器工具包**：用於開發代理的工具，可透過網頁瀏覽或程式碼執行等動作與現實世界互動。
- **提示工程工具包**：一系列資源，協助您撰寫有效提示。

GitLab 工具包總覽

Gitlab 工具包內含多項工具，可讓 LLM 代理與 gitlab 存放區互動。這項工具是 `python-gitlab` 程式庫的包裝函式。

GitLab 工具包可執行下列工作：

- **建立檔案**：在存放區中建立新檔案。
- **讀取檔案**：從存放區讀取檔案。
- **更新檔案**：更新存放區中的檔案。
- **建立提取要求**：從機器人的工作分支版本建立提取要求至基本分支版本。
- **Get Issues**：從存放區擷取問題。
- **取得問題**：擷取特定問題的詳細資料。
- **對問題發表留言**：對特定問題發表留言。
- **刪除檔案**：從存放區刪除檔案。

設定 GitLab 專案

開啟 [GitLab](#)，建立新專案，並在「`Settings / Access Tokens`」下方設定專案存取權杖。

使用下列詳細資料：

- 權杖名稱：`devai-api-qwiklabs`
- 角色：`Maintainer`
- 選取範圍：`api`

Project access tokens

Generate project access tokens scoped to this project for your application HTTP(S). [Learn more.](#)

Active project access tokens  1

Add a project access token

Token name

devai-api-qwiklabs

For example, the application using the token or the purpose of the token

Token description

devai-api-qwiklabs

Expiration date

2026-03-19  

An Administrator has set the maximum expiration date to 2026-03-19.

Select a role

Maintainer 

Select scopes

Scopes set the permission levels granted to the token. [Learn more.](#)

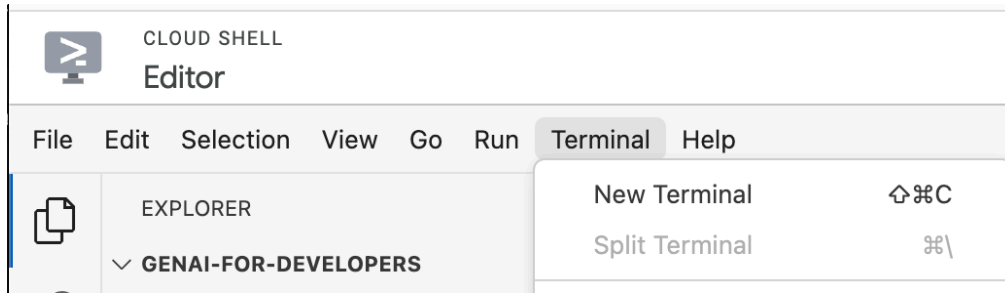
☒ api
Grants complete read and write access to the scoped project API,

將存取權杖值複製並貼到筆電的暫時檔案中，後續步驟會用到。

步驟 6 · 約 0 分鐘

6. 準備在 Cloud Run 上部署應用程式

返回 Cloud Shell 編輯器，使用現有終端機或開啟新的終端機。



使用 Qwiklabs 專案詳細資料設定專案詳細資料。

範例：`qwiklabs-gcp-00-2c10937585bb`

```
gcloud config set project YOUR-QWIKLABS-PROJECT-ID
```

設定其餘環境變數：

```
export PROJECT_ID=$(gcloud config get-value project)
export LOCATION=us-central1
export REPO_NAME=devai-api
export SERVICE_NAME=devai-api
```

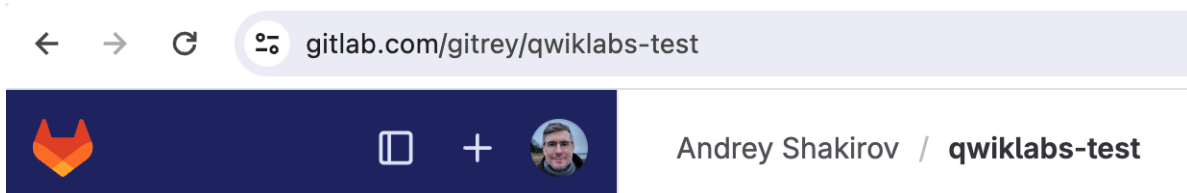
設定 GitLab 整合功能所需的環境變數。

```
export GITLAB_PERSONAL_ACCESS_TOKEN=gitlab-token
```

為避免在終端機中公開私密資訊，最佳做法是使用 `read -s GITLAB_PERSONAL_ACCESS_TOKEN`，這是設定環境變數的安全方式，不會在控制台的指令記錄中顯示值。執行後，您必須貼上該值並按下 Enter 鍵。您也需要匯出環境變數：`export GITLAB_PERSONAL_ACCESS_TOKEN`

這項指令需要您更新 GitLab 使用者 ID 和存放區名稱。

範例：`export GITLAB_REPOSITORY="gitrey/qwiklabs-test"`



```
export GITLAB_REPOSITORY="USERID/REPOSITORY"
```

設定其餘環境變數：


```
export GITLAB_URL="https://gitlab.com"
export GITLAB_BRANCH="devai"
export GITLAB_BASE_BRANCH="main"
```

LangSmith LLM 追蹤設定

如要在 LangSmith 中查看代理程式追蹤資訊，請設定帳戶，否則請直接執行指令。

建立 LangSmith 帳戶，並在「設定」部分產生服務 API 金鑰。 <https://smith.langchain.com/settings>

設定 LangSmith 整合所需的環境變數。

```
export LANGCHAIN_TRACING_V2=true
export LANGCHAIN_ENDPOINT="https://api.smith.langchain.com"

export LANGCHAIN_API_KEY=langchain-service-api-key
```

JIRA 設定

這些值不會在本實驗室中使用，因此您不必在執行指令前，將這些值更新為特定的 JIRA 專案值。

設定 Cloud Run 服務部署作業所需的環境變數。

```
export JIRA_API_TOKEN=jira-token
export JIRA_USERNAME="YOUR-EMAIL"
export JIRA_INSTANCE_URL="https://YOUR-JIRA-PROJECT.atlassian.net"
export JIRA_PROJECT_KEY="YOUR-JIRA-PROJECT-KEY"
export JIRA_CLOUD=true
```

建立 API 金鑰

在 [Cloud 控制台](#)/「API 和服務」/「憑證」專區中，建立新的 API 金鑰。

這組金鑰將用於驗證對 Cloud Run 應用程式的 JIRA API 呼叫。

在正式環境中，您會為建立的 API 金鑰設定應用程式和 API 限制。應用程式限制會規定 API 金鑰只能用於特定網站(例如您的 JIRA 執行個體)。

設定 Cloud Run 服務部署作業所需的環境變數。

```
export DEVAI_API_KEY=your-api-key
```

在 Secret Manager 中建立密鑰

在本實驗室中，我們遵循最佳做法，並使用 [Secret Manager](#) 在 Cloud Run 中儲存及參照存取權杖、API 金鑰和 LangChain API 金鑰值。

在 Secret Manager 中建立及儲存必要密鑰。

```
echo -n $DEVAI_API_KEY | \  
gcloud secrets create DEVAI_API_KEY \  
--data-file=-  
  
echo -n $JIRA_API_TOKEN | \  
gcloud secrets create JIRA_API_TOKEN \  
--data-file=-  
  
echo -n $GITLAB_PERSONAL_ACCESS_TOKEN | \  
gcloud secrets create GITLAB_PERSONAL_ACCESS_TOKEN \  
--data-file=-  
  
echo -n $LANGCHAIN_API_KEY | \  
gcloud secrets create LANGCHAIN_API_KEY \  
--data-file=-
```

步驟 7 · 約 0 分鐘

7. 將 Devai-API 部署到 Cloud Run

確認您位於正確的資料夾。

```
cd ~/genai-for-developers/devai-api
```

將應用程式部署至 Cloud Run。

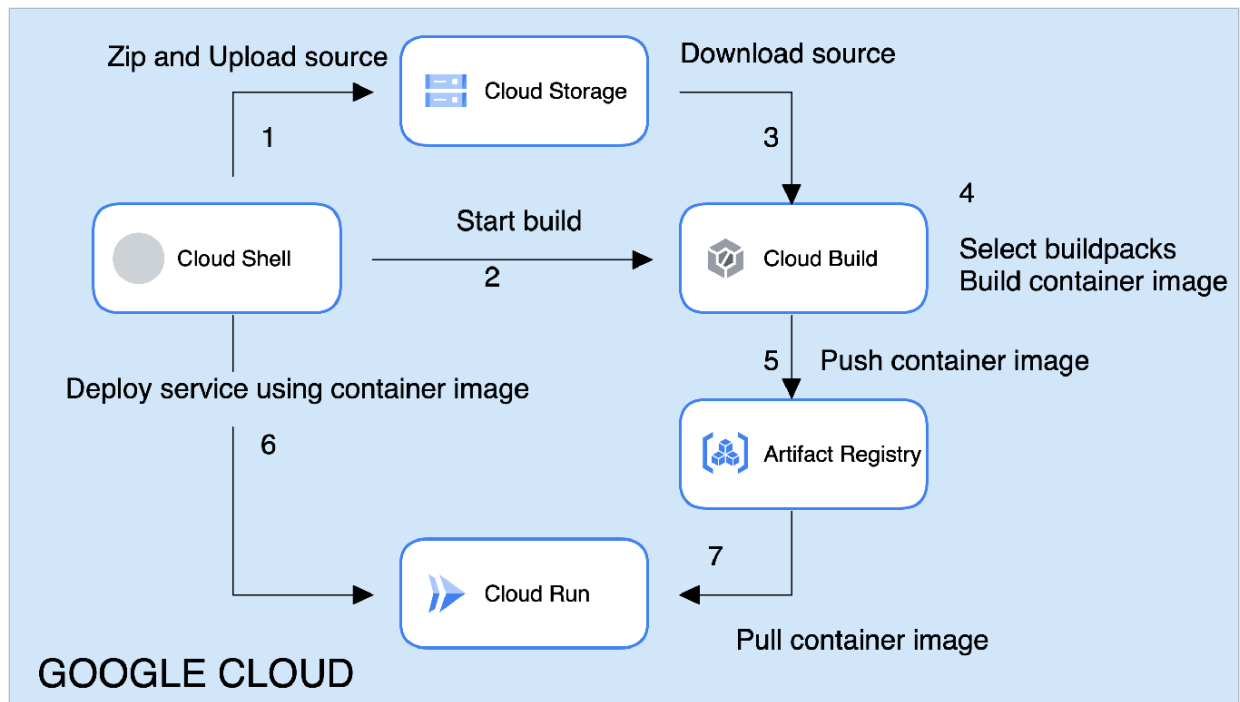
```
gcloud run deploy "$SERVICE_NAME" \
  --source=. \
  --region="$LOCATION" \
  --allow-unauthenticated \
  --service-account vertex-client \
  --set-env-vars PROJECT_ID="$PROJECT_ID" \
  --set-env-vars LOCATION="$LOCATION" \
  --set-env-vars GITLAB_URL="$GITLAB_URL" \
  --set-env-vars GITLAB_REPOSITORY="$GITLAB_REPOSITORY" \
  --set-env-vars GITLAB_BRANCH="$GITLAB_BRANCH" \
  --set-env-vars GITLAB_BASE_BRANCH="$GITLAB_BASE_BRANCH" \
  --set-env-vars JIRA_USERNAME="$JIRA_USERNAME" \
  --set-env-vars JIRA_INSTANCE_URL="$JIRA_INSTANCE_URL" \
  --set-env-vars JIRA_PROJECT_KEY="$JIRA_PROJECT_KEY" \
  --set-env-vars JIRA_CLOUD="$JIRA_CLOUD" \
  --set-env-vars LANGCHAIN_TRACING_V2="$LANGCHAIN_TRACING_V2" \
  --update-secrets="LANGCHAIN_API_KEY=LANGCHAIN_API_KEY:latest" \
  --update-secrets="GITLAB_PERSONAL_ACCESS_TOKEN=GITLAB_PERSONAL_ACCESS_TOKEN:latest" \
  --update-secrets="JIRA_API_TOKEN=JIRA_API_TOKEN:latest" \
  --update-secrets="DEVAI_API_KEY=DEVAI_API_KEY:latest" \
  --min-instances=1 \
  --max-instances=3
```

回答 **y** 建立 Artifact Registry Docker 存放區。

Deploying from source requires an Artifact Registry Docker repository to store built containers. A repository named [cloud-run-source-deploy] in region [us-central1] will be created.

Do you want to continue (Y/n)? **y**

請參閱下方的 `gcloud run deploy SERVICE_NAME --source=.` 流程。[瞭解詳情](#)。



在幕後，這項指令會使用 Google Cloud 的 `buildpacks` 和 `Cloud Build`，從您的原始碼自動建構容器映像檔，而不必在機器上安裝 Docker，也不必設定 Buildpacks 或 Cloud Build。也就是說，上述單一指令會執行原本需要 `gcloud builds submit` 和 `gcloud run deploy` 指令才能執行的作業。

如果您已提供 Dockerfile(我們在這個存放區中就是這麼做)，Cloud Build 就會使用該檔案建構容器映像檔，而不是依賴建構套件自動偵測及建構容器映像檔。如要進一步瞭解建構包，請參閱[說明文件](#)。

在 [控制台](#) 中查看 Cloud Build 記錄。

在 [Artifact Registry](#) 中查看建立的 Docker 映像檔。

在 [Cloud Console](#) 中查看 Cloud Run 執行個體詳細資料。

測試應用程式

執行 `curl` 指令來測試端點。

請先更新 Cloud Run 服務網址，再執行指令。

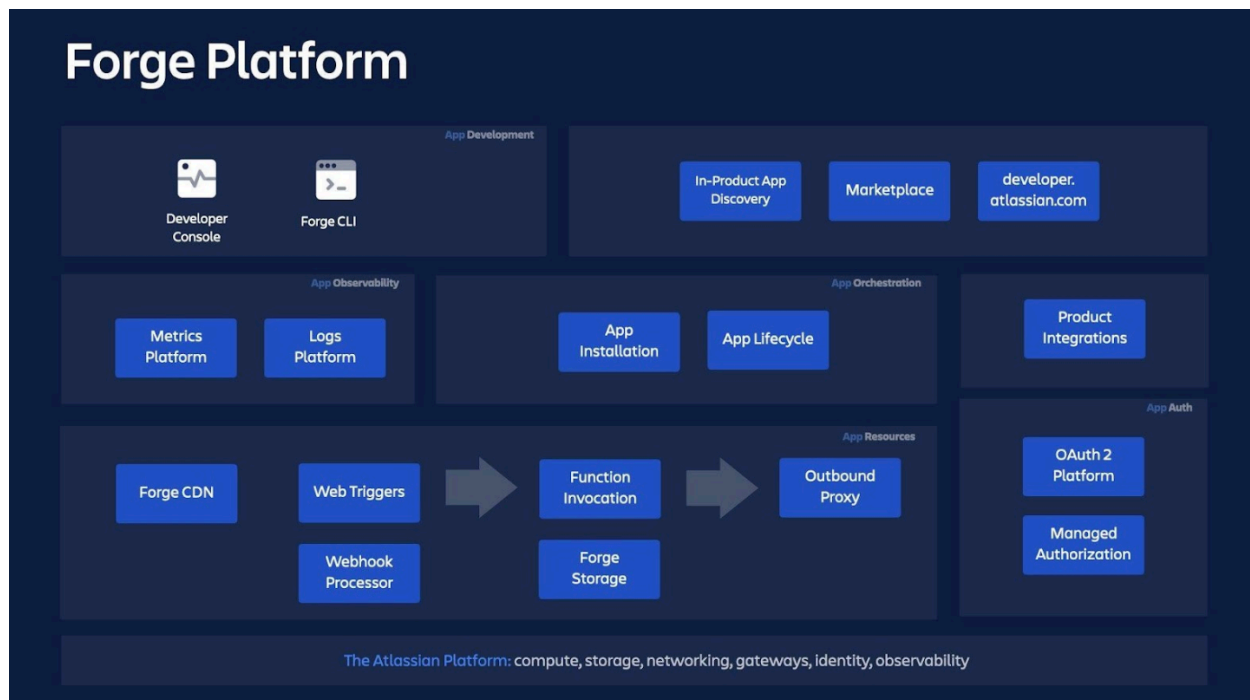
範例：

```
curl -H "X-devai-api-key: $DEVAI_API_KEY" https://devai-api-1110000001.us-central1.run.app/test
```

步驟 8 · 約 0 分鐘

8. 自動導入故事

Forge 平台可讓開發人員建構與 Atlassian 產品 (例如 Jira、Confluence、Compass 和 Bitbucket) 整合的應用程式。

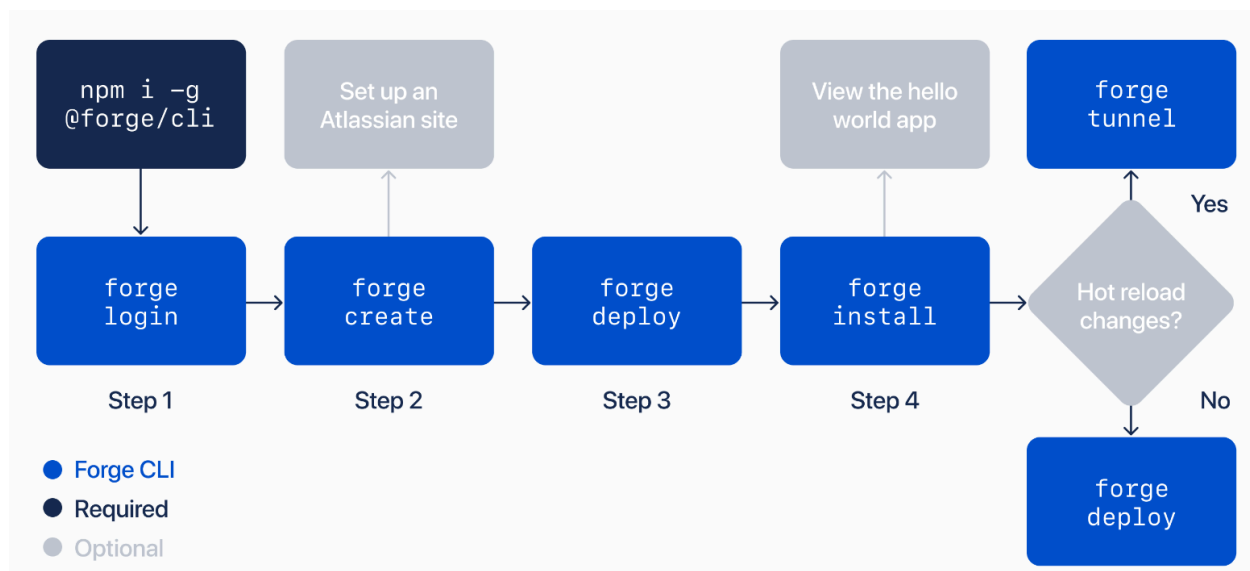


安裝 Forge CLI

執行下列指令，全域安裝 Forge CLI：

```
npm install -g @forge/cli
```

在本實驗室中，我們將使用環境變數登入。



設定 JIRA 專案

使用個人帳戶建立/查看 JIRA 專案。

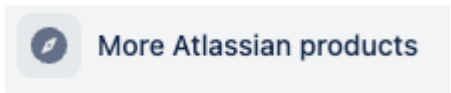
查看現有的 JIRA 專案：<https://admin.atlassian.com/>

使用個人帳戶建立新的 JIRA 專案。

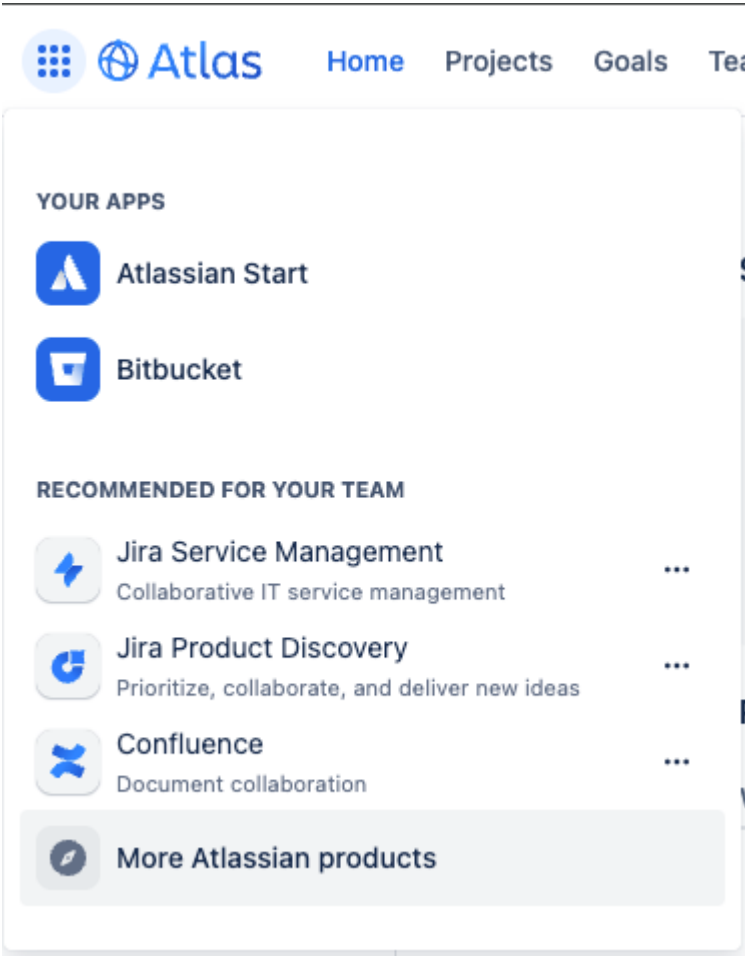
前往 <https://team.atlassian.com/your-work>，然後按一下



，再選取



。然後依序選取「JIRA Software」和「立即試用」。按照提示完成專案/網站建立程序。



選取「JIRA Software」。

Plan, track, and support

◆ Jira Software

Plan, track, and release great software

- Customise your workflow.
- See code and issues together.
- Track your team's progress.

Try it now

[Learn more](#)

建立新專案。

Create project

Explore what's possible when you collaborate with your team. Edit project details anytime in project settings.

Required fields are marked with an asterisk *

Name *

genai4dev

Template

[More templates](#)



Kanban

◆ Jira Software

Visualize and advance your project forward using issues on a powerful board.



[Show more](#)

Back

Create project

建立 Atlassian API 權杖

建立或使用現有的 Atlassian API 權杖登入 CLI。

CLI 執行指令時會使用您的權杖。

1. 前往 <https://id.atlassian.com/manage/api-tokens>。

2. 按一下「Create API Token」(建立 API 憑證)。
3. 輸入用來描述 API 權杖的標籤。例如 `forge-api-token`。
4. 點選「建立」。
5. 按一下「複製到剪貼簿」，然後關閉對話方塊。

設定 Forge 環境

返回 Cloud 控制台，在終端機中執行下列指令。

登入 Forge CLI，即可開始使用 Forge 指令。

設定 JIRA/FORGE 電子郵件地址。請替換成您的電子郵件地址。

```
export FORGE_EMAIL=your-email
```

設定 Forge API 權杖。請替換成您的 JIRA API 權杖。

```
export FORGE_API_TOKEN=your-jira-api-token
```

執行下列指令，測試 `forge` CLI。系統詢問是否要收集數據分析資料時，請回答「`No`」。

```
forge settings set usage-analytics false
```

確認是否已登入。

```
forge whoami
```

輸出範例。

```
Logged in as John Green (johnngreen@email.com)
Account ID: 123090:aaabbcc-076a-455c-99d0-d1aavvcdd
```

建立 Forge 應用程式

確認您位於「`~/genai-for-developers`」資料夾。

執行指令來建立 Forge 應用程式。

```
forge create
```

系統提示時，請使用下列值：

- 應用程式名稱：`devai-jira-ui-qwiklabs`
- 選取類別：`UI Kit`
- 選取產品：`Jira`
- 選取範本：`jira-issue-panel`


```
user@devai-workstation:~/genai-for-developers$ forge create
Creating an app in your current directory:

/home/user/genai-for-developers

Press Ctrl+C to cancel.

Name your app. The app name can include dashes, spaces, and underscores.

? Enter a name for your app: devai-jira-ui-qwiklabs

Start with a template. Each template contains the required files to be a valid app.

? Select a category: UI Kit
? Select a product: Jira
? Select a template: jira-issue-panel

✓ Creating app...

i Downloading template
i Registering app
i Creating environments
i Installing dependencies

✓ Created devai-jira-ui-qwiklabs

Your app is ready to work on, deploy, and install. We created 3 environments you can deploy to: production, staging, development.

Change to directory devai-jira-ui-qwiklabs to see your app files.
```

切換至應用程式資料夾。

```
cd devai-jira-ui-qwiklabs/
```

執行指令來安裝依附元件。

```
npm install
```

執行部署指令。

```
forge deploy
```

輸出內容範例：

```
Deploying your app to the development environment.
Press Ctrl+C to cancel.

Running forge lint...
No issues found.

✓ Deploying devai-jira-ui-qwiklabs to development...

i Packaging app files
i Uploading app
i Validating manifest
i Snapshotting functions
i Deploying to environment

✓ Deployed

Deployed devai-jira-ui-qwiklabs to the development environment.
```

安裝應用程式。

```
forge install
```

系統提示時，請使用下列值：

- 選取產品：Jira
- 輸入網站網址：your-domain.atlassian.net

輸出內容範例：

```
Select the product your app uses.

? Select a product: Jira

Enter your site. For example, your-domain.atlassian.net

? Enter the site URL: genai-for-developers.atlassian.net

Installing your app onto an Atlassian site.
Press Ctrl+C to cancel.

? Do you want to continue? Yes

✓ Install complete!

Your app in the development environment is now installed in Jira on genai-for-developers.atlassian.net
```

開啟 JIRA 網站，並建立新的 JIRA 工作，說明如下：

```
Create HTML, CSS and JavaScript using React.js framework to implement Login page with
username and password fields, validation and documentation. Provide complete implementation,
do not omit anything.
```

開啟工作後，你會看到「devai-jira-ui-qwiklabs」按鈕。

按一下按鈕，然後查看 UI 中的變更。

Projects /  Cymbal Eats /  Add epic /  CYMEATS-477

Login page

Description

-  devai-github
-  devai-gitlab
-  devai-jira-ui-qwiklabs

Create HTML, CSS and JavaScript using React.js framework to implement Login page with username and password fields, validation and documentation. Provide complete implementation, do not omit anything.

查看 Forge 後端記錄。

Atlassian 開發人員控制台

您也可以在 [Atlassian 開發人員控制台](#) 中查看及管理已部署的應用程式。

ATLASSIAN Developer

 **devai-jira-ui-qwiklabs**
Forge App

Overview

BUILD
Environments
Deployments

MONITOR
Platform Health
Metrics
Alerts
Logs
Installations

Console / My apps / devai-jira-ui-qwiklabs

 **devai-jira-ui-qwiklabs**

App details
App ID
a48501f0-938f-4353-848e-f8589fccf4f8
Number of development environments
1
A new development environment is created for each contributor.
Currently deployed to
development

Contributors

Your role	App owner	Total contributors
Admin	Andrey Shakirov	1

查看記錄 - 切換至 **Development** 環境，

ATLASSIAN Developer

 **devai-jira-ui-qwiklabs**
Forge App

Overview

BUILD
Environments
Deployments

MONITOR
Platform Health
Metrics
Alerts
Logs
Installations
Usage

Console / My apps / devai-jira-ui-qwiklabs / Logs

Logs

View app logs to **monitor** and **debug** your Forge apps.

Search for logs by messages, functions, modules, invocation ID, and trace ID

Environment: Development Last 1 hour Site Level More Filters +

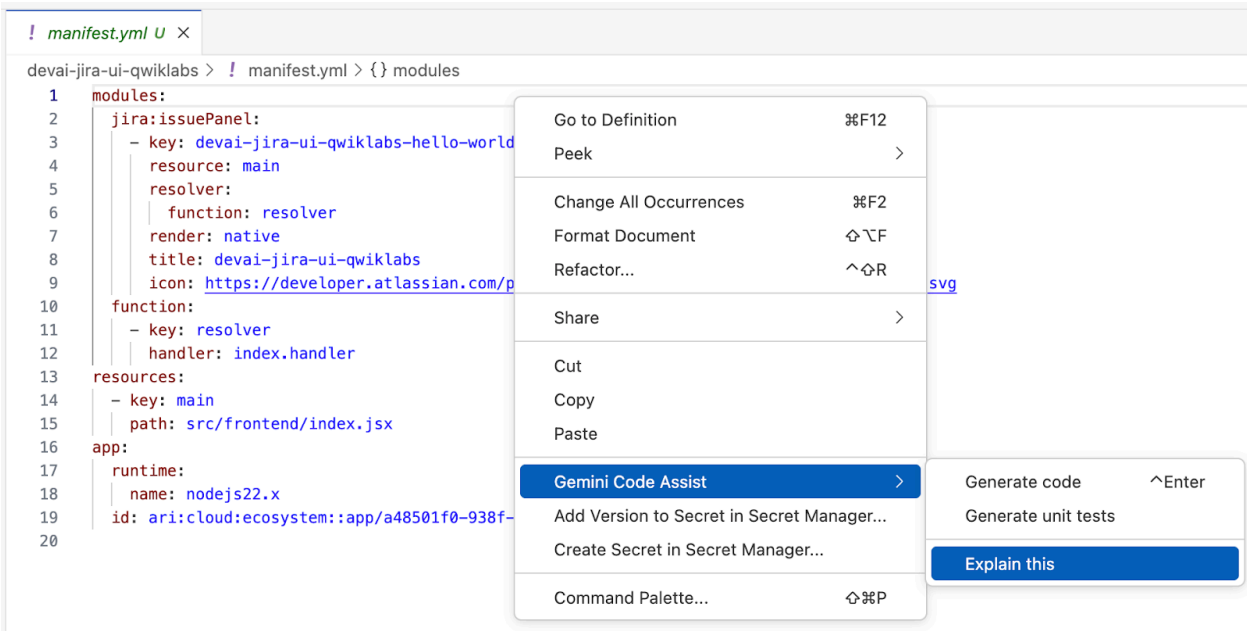
From Mar 19, 2025 15:18 UTC to Mar 19, 2025 16:18 UTC | Last updated a few seconds ago | Currently displ

Level	Date and time (UTC)	Details
> Info	Mar 19, 2025 16:16:43.371	{ payload: { example: 'my-invoke-variable' }, context: { d0c1c563ce3b', localId: 'ari:cloud:ecosystem::extension/i4aef-90b5-977adcbaa243/static/d...
> Info	Mar 19, 2025 16:16:30.091	{ payload: { example: 'my-invoke-variable' }, context: { d0c1c563ce3b', localId: 'ari:cloud:ecosystem::extension/i4aef-90b5-977adcbaa243/static/d...
> Info	Mar 19, 2025 16:16:02.813	{ payload: { example: 'my-invoke-variable' }, context: { d0c1c563ce3b', localId: 'ari:cloud:ecosystem::extension/i4aef-90b5-977adcbaa243/static/d...

No more logs to load

查看 Forge 應用程式資訊清單和原始碼

開啟「devai-jira-ui-qwiklabs/manifest.yml」檔案，然後使用 Gemini Code Assist 解釋設定。



查看說明。

 student 701a20b3

Explain this

 Gemini

Okay, let's break down this `manifest.yml` file. This file is a configuration file for an Atlassian Forge app, specifically one that interacts with Jira. Forge is a serverless platform that allows developers to build apps that extend Atlassian products like Jira and Confluence.

Here's a detailed explanation of each section:

1. modules:

- This section defines the different components or modules that make up your Forge app.
- It's essentially describing the features and functionalities of your app within the Atlassian ecosystem.

a. jira:issuePanel:

- This indicates that this particular module is an "issue panel" in Jira. Issue panels are custom
- ``key: devai-jira-ui-qwiklabs-hello-world-issue-panel`` : This is a unique identifier for this
- ``resource: main``: This connects the issue panel to a specific resource defined later in the
- ``resolver``:
 - ``function: resolver``: This links the issue panel to a resolver function defined in the ``func`
 - ``render: native``: This specifies that the UI for the issue panel will be rendered using Forge
 - ``title: devai-jira-ui-qwiklabs``: This is the title that will be displayed for your issue panel
 - ``icon: https://developer.atlassian.com/platform/forge/images/icons/issue-panel-icon.svg``: This

b. function:

- This section defines the serverless functions that your app uses.
- ``key: resolver``: This is the unique identifier for the resolver function. It matches the one
- ``handler: index.handler``: This specifies the entry point for your function.
 - ``index``: This refers to a file named ``index.js`` (or potentially ``index.ts`` if using TypeScript)
 - ``handler``: This refers to a function named ``handler`` within that ``index`` file. This ``handler``

開啟下列檔案，並要求 Gemini Code Assist 解釋：

- `devai-jira-ui-qwiklabs/src/frontend/index.jsx`
- `devai-jira-ui-qwiklabs/src/resolvers/index.js`



Line by Line Explanation:

1. `import React, { useEffect, useState } from 'react';`
 - This line imports essential components from the React library:
 - `React`: The core React object.
 - `useEffect`: A React Hook for performing side effects in functional components (like fetching data).
 - `useState`: A React Hook for managing state within a functional component.
2. `import ForgeReconciler, { Text } from '@forge/react';`
 - This line is specific to Atlassian Forge:
 - `ForgeReconciler`: This is the renderer for Forge's UI. It's similar to `ReactDOM.render` in a standard React web application, but it's tailored for Forge's environment.
 - `Text`: A component provided by `@forge/react` that renders text in the Forge UI. It likely ensures proper styling and rendering within the Forge context.
3. `import { invoke } from '@forge/bridge';`
 - Also specific to Forge:
 - `invoke`: This function, imported from `@forge/bridge`, is the crucial piece for communicating with the backend of your Forge app. It allows you to call custom functions defined in your Forge backend.
4. `const App = () => { ... };`
 - This defines the main component of your UI, named `App`. It's a functional React component.
5. `const [data, setData] = useState(null);`
 - This uses the `useState` Hook to create a state variable:
 - `data`: This variable will hold the data fetched from the backend. It's initially set to `null`, meaning no data has been loaded yet.
 - `setData`: This is the function used to update the `data` variable.
6. `useEffect(() => { ... }, []);`
 - This uses the `useEffect` Hook. This Hook allows you to perform side effects...

使用 DevAI API Cloud Run 端點更新 Forge 應用程式

檢查是否已設定 GCP 專案 ID：

```
gcloud config get project
```

如果沒有，請使用 Qwiklabs 實驗室頁面中的專案 ID 設定 GCP 專案：

範例：`qwiklabs-gcp-00-2c10937585bb`

```
gcloud config set project YOUR_QWIKLABS_PROJECT_ID
```

設定 Cloud Run 服務網址：

```
export DEVAI_API_URL=$(gcloud run services list --filter="(devai-api)" --format="value(URL)")  
  
forge variables set DEVAI_API_URL $DEVAI_API_URL
```

設定 DEVAI API 金鑰：

```
export DEVAI_API_KEY=api-key-that-you-created  
  
forge variables set --encrypt DEVAI_API_KEY $DEVAI_API_KEY
```

執行下列指令確認：

```
forge variables list
```

輸出範例

```
student_01_ce62a6529c37@cloudshell:~/genai-for-developers/devai-jira-ui-qwiklabs (qw:
Showing variables from the development environment:
```

Encrypted?	Key	Value
✓	DEVAI_API_KEY	****
	DEVAI_API_URL	https://devai-api-133934646591.us-central1.run.app

Note, the values of encrypted variables are shown as ****.

更新 Forge 應用程式資訊清單和程式碼

這些程式碼片段位於存放區的 `sample-devai-jira-ui` 資料夾中。

在編輯器中開啟資訊清單檔案：`devai-jira-ui-qwiklabs/manifest.yml`

在檔案結尾處加上以下幾行，並將 Cloud Run 端點替換為您部署的端點。

```
permissions:
  scopes:
    - read:jira-work
    - write:jira-work
  external:
    fetch:
      client:
        - devai-api-gjerpi6qqq-uc.a.run.app/create-gitlab-mr # replace with YOUR CLOUD RUN
        URL
```

在編輯器中開啟 `resolvers/index` 檔案：`devai-jira-ui-qwiklabs/src/resolvers/index.js`

在現有 `getText` 函式下方新增指令行。

```
resolver.define('getApiKey', (req) => {
  return process.env.DEVAI_API_KEY;
});

resolver.define('getDevAIApiUrl', (req) => {
  return process.env.DEVAI_API_URL;
});
```

在編輯器中開啟 `frontend/index` 檔案：`devai-jira-ui-qwiklabs/src/frontend/index.jsx`

將 `index.jsx` 替換為下列內容。更新 GitLab 使用者 ID/存放區的連結。

您需要在兩個位置更新 `YOUR-GIT-USERID` 和 `YOUR-GIT-REPO`。

在檔案中搜尋這一行，然後進行變更：

```
https://gitlab.com/ YOUR-GIT-USERID/YOUR-GIT-REPO /-/merge_requests
```

```

import React from 'react';
import ForgeReconciler, { Text, Link, useProductContext } from '@forge/react';
import { requestJira } from '@forge/bridge';
import { invoke } from '@forge/bridge';
import api, { route, assumeTrustedRoute } from '@forge/api';

const devAIApiKey = await invoke("getApiKey")
const devAIApiUrl = await invoke("getDevAIApiUrl")

const App = () => {
  const context = useProductContext();

  const [description, setDescription] = React.useState();

  const fetchDescriptionForIssue = async () => {
    const issueId = context?.extension.issue.id;

    const res = await requestJira(`/rest/api/2/issue/${issueId}`);
    const data = await res.json();

    const bodyGenerateData = `{"prompt": ${JSON.stringify(data.fields.description)}}`;

    const generateRes = await api.fetch(devAIApiUrl+'/create-gitlab-mr,
    {
      body: bodyGenerateData,
      method: 'post',
      headers: {
        'Content-Type': 'application/json',
        'x-devai-api-key': devAIApiKey,
      },
    },
  )

  const resData = await generateRes.text();

  // Add link to the GitLab merge request page as a comment
  await requestJira(`/rest/api/2/issue/${issueId}/comment`, {
    method: 'POST',
    headers: {
      'Accept': 'application/json',
      'Content-Type': 'application/json'
    },
    body: `{"body": "[GitLab Merge Request|https://gitlab.com/YOUR-GIT-USERID/YOUR-GIT-REPO/-/merge_requests]}"`
  });

  return "Response will be added as a comment. Please refresh in a few moments.";
};

React.useEffect(() => {

```



```

    if (context) {
      fetchDescriptionForIssue().then(setDescription);
    }
  }, [context]);

  return (
    <>
      <Text>{description}</Text>
      <Link href='https://gitlab.com/YOUR-GIT-USERID/YOUR-GIT-REPO/-/merge_requests'
openNewTab={true}>GitLab Merge Request</Link>
    </>
  );
};

ForgeReconciler.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

```

重新部署 Forge 應用程式

在 `package.json` 檔案中新增依附元件：

```
"@forge/api": "4.0.0",
```

執行指令來安裝依附元件：

```
npm install
```

部署更新後的應用程式：

```
forge deploy
```

輸出內容範例：

```

i Uploading app
i Validating manifest
i Snapshotting functions
i Deploying to environment

```

```
✓ Deployed
```

Deployed devai-jira-ui-qwiklabs to the development environment.

We've detected new scopes or egress URLs in your app.

Run `forge install --upgrade` and restart your tunnel to put them into effect.

安裝更新後的應用程式：

```
forge install --upgrade
```

```
user@devai-workstation:~/github/genai-for-developers/devai-jira-ui-qwiklabs$ forge install --upgrade
To upgrade your app to use the latest scopes, select it from the list.
Press Ctrl+C to cancel.
? Select the site or workspace to upgrade: (Use the Enter key to select)
```

Environment	Site	Product	Version
> development	genai-for-developers.atlassian.net	Jira	Out-of-date

輸出內容範例：

```
Upgrading your app on the Atlassian site.

Your app will be upgraded with the following additional scopes:
- read:jira-work
- write:jira-work

Your app will exchange data with the following urls:
- devai-api-7su2ctuqpq-uc.a.run.app

? Do you want to continue? Yes

✓ Upgrade complete!

Your app in the development environment is now the latest in Jira on genai-for-developers.atlassian.net.
```

測試 Forge 應用程式

在 JIRA 專案中開啟現有或建立新的 JIRA 工作。

如果先前已新增面板，請先移除。

按一下「...」，然後從選單中選取「移除」。之後，你就可以再次點選該按鈕。

Login page

+ Add

📁 Apps

Description

Create HTML, CSS and JavaScript using React.js framework to implement Login page with username and password fields, validation and documentation. Provide complete implementation, do not omit anything.

devai-jira-ui-qwiklabs DEV



Actions

Collapse

Remove

Activity

查看 Jira 留言

收到 DEVAI API 的回覆後，系統會在 JIRA 問題中新增註解。

- GitLab 合併要求連結

在「History」和「Comments」分頁之間切換，即可重新整理檢視畫面。

devai-jira-ui-qwiklabs DEV

Response will be added as a comment. Please refresh in a few moments.
[GitLab Merge Request](#)

Activity

Show: All Comments History Work log



Add a comment...



Looks good!



Need help?



This is blocked...

Pro tip: press **M** to comment



Andrey Shakirov 34 seconds ago

[GitLab Merge Request](#)

驗證 GitLab 合併要求

開啟 GitLab，並檢查專案中是否有新的合併要求。

Project

devai-api-qwiklabs

Pinned

Issues

Merge requests

Manage

Plan

Project

devai-api-qwiklabs

Pinned

Issues

Merge requests

Manage

Plan

Code

Merge requests

Repository

Branches

Commits

Tags

Repository graph

Overview

Commits

Pipelines

Changes

This pull request implements a basic login page with username and password fields. It includes input validation and displays error messages for empty fields. The changes include:

- Creation of LoginPage.js component
- Creation of LoginPage.css for styling
- Modification of App.js to include the LoginPage component

👍 0

👎 0

😊

Files

src

App.js

LoginPage.css

LoginPage.js

src/LoginPage.css

src/LoginPage.js

```
1 + import React, { useState } from 'react';
2 + import './LoginPage.css';
3 +
4 + function LoginPage() {
5 +   const [username, setUsername] = useState('');
6 +   const [password, setPassword] = useState('');
7 +   const [usernameError, setUsernameError] = useState('');
8 +   const [passwordError, setPasswordError] = useState('');
9 +
10 +   const handleUsernameChange = (event) => {
11 +     setUsername(event.target.value);
12 +     setUsernameError(''); // Clear error on input
13 +   };
14 +
15 +   const handlePasswordChange = (event) => {
16 +     setPassword(event.target.value);
17 +     setPasswordError(''); // Clear error on input
18 +   };
19 +
```

LangSmith LLM 追蹤記錄

如果您已設定 LLM 追蹤，請開啟 [LangSmith 入口網站](#)，並查看 GitLab 合併要求建立呼叫的 LLM 追蹤記錄。

LangSmith 大型語言模型追蹤記錄範例。

TRACE

Collapse Stats Show All

AgentExecutor 99.71s 15,118 zero-shot-react-description

- LLMChain 34.76s
- ChatVertexAI 34.75s
- Create Pull Request 0.99s
- LLMChain 23.06s
- ChatVertexAI 23.06s
- Create File 0.75s
- LLMChain 15.13s
- ChatVertexAI 15.12s
- Create File 1.22s
- LLMChain 10.78s
- ChatVertexAI 10.77s
- Create File 0.68s
- LLMChain 9.38s
- ChatVertexAI 9.37s
- Create File 0.22s

AgentExecutor

Run ID Trace ID

Run Feedback Metadata

Input

```
1 input: "Create GitLab merge request using provided details below.
2
3 \   Create new files, commit them and push them to opened merge request.
4
5 \   When creating new files, remove the lines that start with ``` before saving the
6 files.
7
8   GITLAB_PERSONAL_ACCESS_TOKEN
9
10 \   DETAILS:\
11
12 \   ## Implementing a Login Page with React.js:
13
14   Here's a comprehensive guide to building a login page with React.js, incorporating
15   username and password fields, validation, and documentation:
16
17
18   **1. Setting Up the Project:**
19
20   *   **Create React App:** Use `create-react-app` to quickly set up a new React
21   project. Open your terminal and run:
22
23   ```bash
24
25   npx create-react-app login-page
26
27   cd login-page
28
```

YAML

Output

```
1 output: I have successfully created a pull request with the title "Implementing a
2 Login Page with React.js" and added the necessary files (`src/LoginForm.js`,
3 `src/App.js`, and `src/styles.css`) to the merge request. The `README.md` file already
4 existed, so I did not create a duplicate.
5
6 intermediate_steps:
7   - tool: Create Pull Request
8     tool_input: |-
9       Implementing a Login Page with React.js
10
11       Here's a comprehensive guide to building a login page with React.js,
12       incorporating username and password fields, validation, and documentation:
13
14       **1. Setting Up the Project:**
15
16       *   **Create React App:** Use `create-react-app` to quickly set up a new React
17       project. Open your terminal and run:
```

步驟 9 · 約 0 分鐘

9. 將變更推送至 GitHub 存放區

(OPTIONAL SECTION)

前往 [GitHub](#) 網站，建立新的存放區，將本實驗室的變更推送到個人存放區。

返回 Cloud Shell，在終端機中設定 Git 使用者名稱和電子郵件地址。

請先更新值，再執行指令。

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

產生安全殼層金鑰，並新增至 GitHub 存放區。

請先更新電子郵件地址，再執行指令。

請勿輸入通關密語，並多次按下 Enter 鍵來完成金鑰產生程序。

```
ssh-keygen -t ed25519 -C "your-email-address"
```

```
eval "$(ssh-agent -s)"

ssh-add ~/.ssh/id_ed25519

cat ~/.ssh/id_ed25519.pub
```

將產生的公開金鑰新增至 GitHub 帳戶。

開啟 <https://github.com/settings/keys>，然後按一下「New SSH key」。

金鑰名稱請使用「`qwiklabs-key`」，並複製/貼上上一個指令的輸出內容。

返回終端機，修訂並推送變更。

```
cd ~/genai-for-developers

git remote rm origin
```

使用上述建立的存放區設定遠端來源。

請替換為您的存放區網址。

```
git remote add origin git@github.com:YOUR-GITHUB-USERID/YOUR-GITHUB-REPO.git
```

新增、修訂及推送變更。

```
git add .
```

```
git commit -m "lab changes"
```

```
git push -u origin main
```

步驟 10 · 約 0 分鐘

10. 恭喜！

恭喜，您已完成本實驗室！

涵蓋內容：

- 如何部署 Cloud Run 應用程式，與 Gemini API 整合。
- 如何建立及部署 JIRA 適用的 Atlassian Forge 應用程式。
- 如何使用 LangChain ReAct 代理程式自動執行 GitLab 工作。
- 如何在 LangSmith 中查看 LLM 追蹤記錄。

後續步驟：

- 更多實務課程即將推出！

清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本教學課程所用資源的費用，請刪除含有相關資源的專案，或者保留專案但刪除個別資源。

刪除專案

如要避免付費，最簡單的方法就是刪除您為了本教學課程所建立的專案。

©2024 Google LLC 保留所有權利。Google 和 Google 標誌是 Google LLC 的商標，所有其他公司和產品名稱可能是其關聯公司的商標。
